



Desarrollo Cloud Native

Evaluación Final Transversal

Índice

1. Descripción general de la solución	4
2. Objetivo y alcance.....	5
3. Arquitectura implementada	6
4. Estructura del repositorio y microservicios.....	6
4.1 Estructura general del repositorio.....	7
4.2 Organización interna de los microservicios	8
4.3 BFF Service.....	8
4.4 Cursos Service	9
4.5 Inscripciones Service	10
4.6 Contenedores y comunicación interna	10
4.7 Beneficios de la estructura utilizada	11
5. Persistencia en Oracle Autonomous Database.....	12
6. Configuración de IDaaS (Azure AD B2C)	13
6.1 Creación y selección del tenant.....	14
6.2 Registro de la aplicación	14
6.3 Flujo de usuario y emisión de JWT	15
7. Seguridad del backend con Spring Security	16
8. Configuración de AWS API Gateway	17
8.1 Creación del autorizador JWT	17
8.2 Registro de rutas e integración.....	18
9. Configuración de RabbitMQ y Docker.....	19
10. Productor, consumidor y Dead Letter Queue	22
10.1 Endpoint productor	22
10.2 Consumidor automático	22

10.3 Flujo de error y DLQ.....	22
11. Almacenamiento Cloud con Amazon S3	24
11.1 Creación y configuración del bucket.....	24
12. Pipeline CI/CD y despliegue en Amazon EC2	26
13. Validación funcional con Postman	27
14. Catálogo de endpoints	32
15. Conclusiones y repositorio	33

1. Descripción general de la solución

Se desarrolló una plataforma Cloud Native para la gestión de cursos en línea, compuesta por tres microservicios implementados con Spring Boot: un BFF encargado de centralizar y orquestar las solicitudes, un servicio de cursos y un servicio de inscripciones. La solución integra Azure AD B2C como servicio de identidad para autenticación y autorización, AWS API Gateway como punto de entrada público, RabbitMQ desplegado en Docker para el procesamiento de mensajería asíncrona, Amazon S3 para el almacenamiento de materiales, Oracle Autonomous Database para la persistencia de datos y GitHub Actions para automatizar la construcción, publicación y despliegue de los servicios en Amazon EC2. La plataforma permite que los estudiantes consulten cursos, realicen inscripciones, descarguen materiales, rindan exámenes y revisen sus calificaciones. Por su parte, los instructores pueden crear, actualizar y desactivar cursos, administrar archivos, crear y modificar exámenes, revisar o ajustar calificaciones y consumir mensajes almacenados en la cola de errores de RabbitMQ. Todos los endpoints públicos se encuentran protegidos mediante tokens JWT y reglas de autorización basadas en los roles ESTUDIANTE e INSTRUCTOR.

El proyecto se encuentra desplegado y operativo en AWS. Las pruebas funcionales se realizaron desde Postman utilizando la URL pública de API Gateway y tokens emitidos por Azure AD B2C. Durante la validación se comprobaron respuestas 401 Unauthorized para solicitudes sin autenticación y 403 Forbidden para operaciones no permitidas según el rol, además de operaciones CRUD sobre cursos, carga, listado, reemplazo, descarga y eliminación de archivos en Amazon S3, procesamiento asíncrono de inscripciones con RabbitMQ, envío de mensajes a una cola de errores mediante DLQ, consumo manual de dichos mensajes y gestión completa de exámenes y calificaciones.

La implementación responde a los requerimientos de la Evaluación Final Transversal, ya que incorpora almacenamiento en la nube, Identity as a Service, API Manager, colas de mensajes, persistencia en Oracle y despliegue automatizado. Asimismo, el repositorio contiene el código fuente de los microservicios, los scripts para la creación de las tablas, la

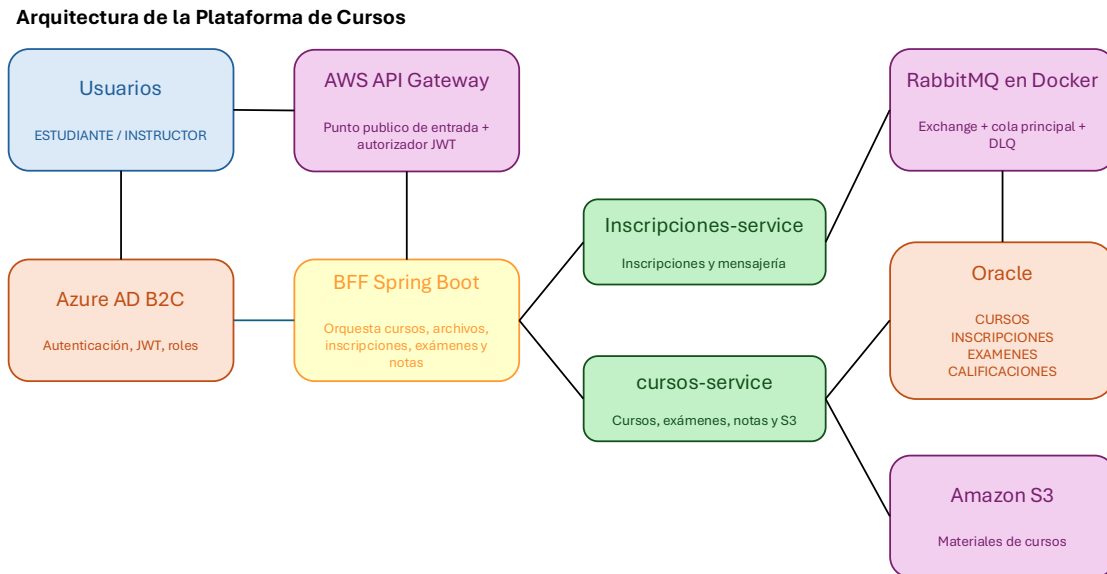
configuración de RabbitMQ, los archivos Docker y el pipeline de integración y despliegue continuo.

2. Objetivo y alcance

El objetivo fue construir una plataforma de cursos desacoplada, segura y desplegable, aplicando patrones propios de una arquitectura Cloud Native. La solución debía satisfacer los requerimientos de estudiantes e instructores, devolver información JSON, proteger el backend, integrar almacenamiento Cloud y utilizar al menos un endpoint productor y otro consumidor de mensajes.

- **ESTUDIANTE:** consulta cursos, se inscribe, lista y descarga materiales, responde exámenes y consulta sus calificaciones.
- **INSTRUCTOR:** crea, actualiza y desactiva cursos; administra archivos; crea y modifica exámenes; revisa y ajusta calificaciones; consume mensajes de la cola de errores.
- **BFF:** concentra la seguridad y orquesta las llamadas hacia cursos-service e inscripciones-service.
- **Persistencia:** Oracle Autonomous Database mediante wallet montado en los contenedores.
- **Despliegue:** imágenes Docker publicadas en Docker Hub y ejecutadas en Amazon EC2 mediante GitHub Actions.

3. Arquitectura implementada



Las solicitudes ingresan por AWS API Gateway. El autorizador JWT valida el token emitido por Azure AD B2C y luego la integración redirige la solicitud al BFF desplegado en EC2. El BFF aplica reglas adicionales de Spring Security y comunica la operación al microservicio correspondiente. `cursos-service` accede a Oracle y Amazon S3, mientras `inscripciones-service` utiliza Oracle y RabbitMQ. GitHub Actions automatiza pruebas, construcción de imágenes y despliegue.

4. Estructura del repositorio y microservicios

Se utilizó una estructura de tipo **monorepositorio**, en la cual los tres microservicios, los scripts de base de datos, los archivos de contenedores y el pipeline de despliegue se mantienen dentro de un mismo proyecto. Esta organización facilita la administración de versiones, la ejecución de pruebas y el despliegue coordinado de todos los componentes.

4.1 Estructura general del repositorio

La estructura principal del proyecto es la siguiente:

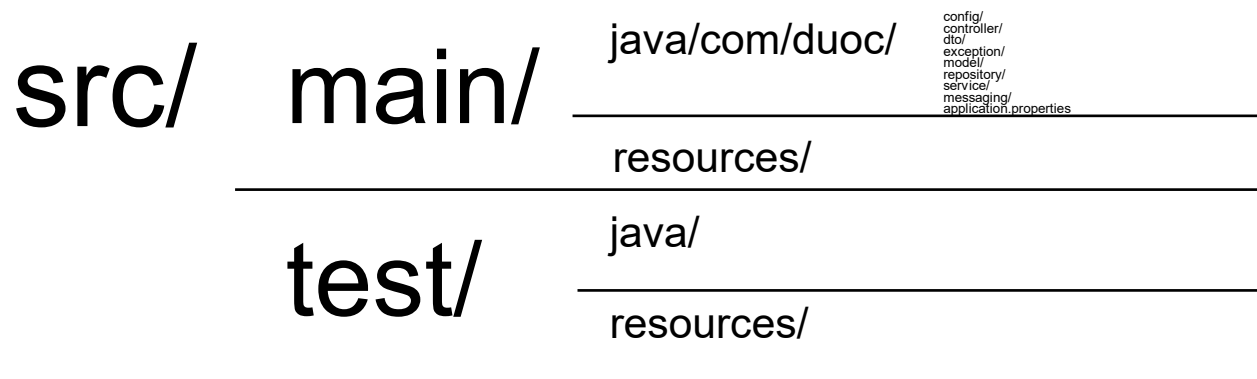
plataforma- cursos- cloud	.github/	workflows/	main.yml
	bff-service/	src/ pom.xml Dockerfile mvnw mvnw.cmd	
	cursos-service/	src/ pom.xml Dockerfile mvnw mvnw.cmd	
	inscripciones-service/	src/ pom.xml Dockerfile mvnw mvnw.cmd	
	database/	01_crear_tablas.sql 02_insertar_datos.sql 03_crear_examenes_calificaciones.sql	
	documentacion/		
	oracle/		
	rabbitmq/		
	docker- compose.prod.yml		
	README.md		
	.gitignore		
	.gitattributes		

En la raíz se encuentran las carpetas correspondientes a los tres microservicios principales: bff-service, cursos-service e inscripciones-service. También se incluyen carpetas auxiliares para los scripts de Oracle, documentación, recursos de autenticación e integración con RabbitMQ. El archivo docker-compose.prod.yml contiene la definición de los contenedores utilizados en producción, mientras que .github/workflows/main.yml contiene el pipeline de integración y despliegue continuo.

Cada microservicio corresponde a un proyecto Maven independiente y contiene su propio archivo pom.xml, Maven Wrapper, código fuente, pruebas y Dockerfile. Esto permite compilar, probar y construir la imagen Docker de cada componente de manera independiente.

4.2 Organización interna de los microservicios

Los servicios fueron desarrollados utilizando una arquitectura por capas. La estructura interna general es la siguiente:



La capa controller expone los endpoints REST; la capa service contiene la lógica de negocio; repository administra el acceso a Oracle mediante Spring Data JPA; model contiene las entidades persistentes; dto define los objetos utilizados en las solicitudes y respuestas; config concentra las configuraciones de seguridad, Amazon S3 o RabbitMQ; y exception administra las respuestas de error. La carpeta messaging se utiliza principalmente en el servicio de inscripciones para el productor y consumidor de mensajes.

4.3 BFF Service

El microservicio bff-service actúa como **Backend for Frontend** y constituye el punto central de acceso a la aplicación. Recibe las solicitudes provenientes de AWS API Gateway y las redirige al microservicio correspondiente.

Este servicio se ejecuta en el puerto 8080 y utiliza la ruta base: **/api/plataforma**

Sus principales responsabilidades son:

- Centralizar los endpoints públicos de la plataforma.
- Validar los tokens JWT emitidos por Azure AD B2C.
- Aplicar las reglas de autorización para los roles ESTUDIANTE e INSTRUCTOR.
- Comunicarse con cursos-service e inscripciones-service.
- Unificar las respuestas y el manejo de errores.
- Transferir solicitudes JSON y archivos multipart.
- Evitar que los microservicios internos sean expuestos directamente a Internet.

El BFF no posee una base de datos propia. Su función es orquestar las solicitudes utilizando RestClient, enviándolas al servicio de cursos o al servicio de inscripciones. Además, Spring Security se encuentra configurado en este componente como OAuth2 Resource Server, por lo que todas las solicitudes protegidas deben presentar un token JWT válido.

4.4 Cursos Service

El microservicio cursos-service se ejecuta internamente en el puerto 8081 y concentra la lógica relacionada con cursos, materiales, exámenes y calificaciones.

Sus principales funcionalidades son:

- Listar y consultar cursos.
- Crear, actualizar y desactivar cursos.
- Subir materiales a Amazon S3.
- Listar, reemplazar, descargar y eliminar archivos.
- Crear, consultar, actualizar y desactivar exámenes.
- Registrar respuestas de estudiantes.
- Calcular y almacenar calificaciones.
- Consultar calificaciones por estudiante o por curso.
- Permitir que un instructor ajuste manualmente un puntaje.

La información estructurada se almacena en Oracle Autonomous Database mediante Spring Data JPA. Para la administración de archivos se utiliza AWS SDK for Java, enviando los materiales a un bucket de Amazon S3.

El servicio mantiene separadas las responsabilidades mediante controladores como CursoController, ArchivoController, ExamenController y CalificacionController. Los controladores delegan la lógica en sus respectivos servicios y repositorios. El CRUD de cursos utiliza métodos GET, POST, PUT y DELETE, mientras que la eliminación corresponde a una desactivación lógica del registro.

4.5 Inscripciones Service

El microservicio inscripciones-service se ejecuta internamente en el puerto 8082 y administra el proceso de inscripción de estudiantes.

Sus principales responsabilidades son:

- Crear y consultar inscripciones.
- Listar inscripciones por estudiante o por curso.
- Persistir inscripciones en Oracle Autonomous Database.
- Publicar solicitudes de inscripción en RabbitMQ.
- Consumir automáticamente mensajes desde la cola principal.
- Enviar a una cola de errores los mensajes que no puedan ser procesados.
- Permitir el consumo manual de mensajes desde la cola de errores.

La comunicación asíncrona se implementó mediante Spring AMQP. El productor publica mensajes utilizando RabbitTemplate, mientras que el consumidor utiliza la anotación `@RabbitListener`. Cuando un mensaje contiene la propiedad `forzarError` con valor verdadero, el procesamiento genera una excepción y RabbitMQ lo redirige mediante un Dead Letter Exchange hacia `inscripciones.error.queue`.

Este mecanismo permite desacoplar la recepción de la solicitud del procesamiento definitivo de la inscripción. El endpoint responde inmediatamente con estado `EN_PROCESO`, mientras que el consumidor procesa la operación posteriormente y registra la inscripción en Oracle.

4.6 Contenedores y comunicación interna

Cada microservicio posee su propio Dockerfile y genera una imagen independiente. En el ambiente de producción se ejecutan cuatro contenedores:

- `plataforma-bff-service`
- `plataforma-cursos-service`
- `plataforma-inscripciones-service`
- `plataforma-rabbitmq`

Los contenedores se conectan mediante una red Docker denominada plataforma-network. El BFF se comunica con los servicios internos utilizando los nombres de los contenedores:

- `http://cursos-service:8081`
- `http://inscripciones-service:8082`

Solo el puerto 8080 del BFF se publica para recibir solicitudes de la plataforma. Los puertos 8081 y 8082 permanecen disponibles únicamente dentro de la red Docker. RabbitMQ utiliza internamente el puerto 5672, mientras que su consola de administración se publica en el puerto 15672 para efectos de monitoreo y demostración.

4.7 Beneficios de la estructura utilizada

La separación en microservicios permite que cada componente tenga una responsabilidad definida. El BFF concentra la seguridad y el acceso público; el servicio de cursos administra el contenido académico y los archivos; y el servicio de inscripciones administra la persistencia y la comunicación asíncrona.

Esta estructura entrega los siguientes beneficios:

- Separación clara de responsabilidades.
- Compilación y construcción independiente por servicio.
- Mayor facilidad para realizar pruebas.
- Aislamiento de errores entre componentes.
- Posibilidad de escalar cada servicio de manera independiente.
- Centralización de la seguridad en el BFF.
- Automatización del despliegue mediante GitHub Actions.
- Administración coordinada mediante Docker Compose.

En conjunto, la estructura del repositorio refleja una solución modular y preparada para ser desplegada en un entorno Cloud Native.

5. Persistencia en Oracle Autonomous Database

La base de datos se configura mediante variables de entorno para usuario y contraseña, junto con un wallet de Oracle montado de forma privada en /app/wallet. El pipeline recibe el wallet codificado en Base64, valida su estructura, lo transfiere a EC2 y lo monta en los contenedores que requieren conexión a Oracle.

Script	Contenido
database/01_crear_tablas.sql	Creación de CURSOS e INSCRIPCIONES, claves primarias, restricciones, índices y relación entre tablas.
database/02_insertar_datos.sql	Datos iniciales para validar cursos e inscripciones.
database/03_crear_examenes_calificaciones.sql	Creación de EXAMENES y CALIFICACIONES, restricciones, índices y relaciones.

Tabla	Uso principal
CURSOS	Información del curso, instructor, cupo y estado activo.
INSCRIPCIONES	Estudiante, curso, estado y fecha de inscripción.
EXAMENES	Curso, título, pregunta, respuesta correcta, puntaje máximo y estado.
CALIFICACIONES	Respuesta del estudiante, puntaje y fecha de respuesta.

The screenshot shows the Oracle Database Actions | SQL interface. On the left, the 'Navegador' (Navigator) pane shows a tree structure with 'ADMIN' and 'Tablas' (Tables) expanded. The 'Tablas' pane lists four tables: CALIFICACIONES, CURSOS, DBTOOLS\$EXECUTION_HISTORY, EXAMENES, and INSCRIPCIONES. The main SQL editor displays the following query:

```

1 SELECT
2   object_name AS tabla,
3   status,
4   TO_CHAR(created, 'DD-MM-YYYY HH24:MI:SS') AS fecha_creacion,
5   TO_CHAR(last_ddl_time, 'DD-MM-YYYY HH24:MI:SS') AS ultima_modificacion
6 FROM user_objects
7 WHERE object_type = 'TABLE'
8    AND object_name IN (
9     'CURSOS',
10    'INSCRIPCIONES',
11    'EXAMENES',
12    'CALIFICACIONES'
13 )
14 ORDER BY CASE object_name
15   WHEN 'CURSOS' THEN 1
16   WHEN 'INSCRIPCIONES' THEN 2
17   WHEN 'EXAMENES' THEN 3
18   WHEN 'CALIFICACIONES' THEN 4
19 END;

```

Below the SQL editor, the 'Resultado de la Consulta' (Query Result) pane shows the execution results. The execution time is 0.038 segundos. The results are displayed in a table with the following columns: TABLA, STATUS, FECHA_CREACION, and ULTIMA_MODIFICA.

	TABLA	STATUS	FECHA_CREACION	ULTIMA_MODIFICA
1	CURSOS	VALID	11-07-2026 19:10:...	11-07-2026 19:10:...
2	INSCRIPCIONES	VALID	11-07-2026 19:10:...	11-07-2026 19:10:...
3	EXAMENES	VALID	12-07-2026 19:38:...	12-07-2026 19:38:...
4	CALIFICACIONES	VALID	12-07-2026 19:38:...	12-07-2026 19:38:...

Tablas creadas en Oracle Autonomous Database. La evidencia muestra las tablas CURSOS, INSCRIPCIONES, EXAMENES y CALIFICACIONES utilizadas para almacenar la información principal de la plataforma. Todas las tablas se encuentran disponibles y con estado válido.

6. Configuración de IDaaS (Azure AD B2C)

Azure AD B2C se utilizó como servicio IDaaS para administrar usuarios y emitir tokens JWT. La configuración se realizó sobre el tenant duocplataformacursos y se integró tanto con Postman como con el backend Spring Boot.

6.1 Creación y selección del tenant

1. Crear o seleccionar el tenant Azure AD B2C asociado al proyecto.
2. Configurar el nombre de organización duocplataformacursos, dominio inicial y región.
3. Ingresar al directorio B2C antes de registrar las aplicaciones.

Básico	
Tipo de inquilino	Azure AD B2C
Configuración	
Nombre de la organización	duocplataformacursos
Nombre de dominio inicial	duocplataformacursos.onmicrosoft.com
País o región	Estados Unidos
Suscripción	Azure for Students
Grupo de recursos	duocplataformacursos
Ubicación del grupo de recursos	East US 2

Información general del tenant Azure AD B2C utilizado por la plataforma.

6.2 Registro de la aplicación

4. Ingresar a Azure AD B2C > Registros de aplicaciones > Nuevo registro.
5. Registrar la aplicación de API con el nombre plataforma-cursos-api.
6. Configurar una URI de redirección para las pruebas. Durante la validación se utilizó `https://jwt.ms`
7. Registrar el identificador de cliente, utilizado posteriormente como audiencia del JWT.

Todos los servicios > Azure AD B2C | Registros de aplicaciones

Registrar una aplicación

* Nombre

Nombre para mostrar de esta aplicación. Se puede cambiar posteriormente.

plataforma-cursos-api

Tipos de cuenta compatibles

Elija los tipos de cuenta que podrán usar esta aplicación o acceder a esta API.

Cualquier inquilino de Entra ID + cuentas personales de Microsoft

URI de redirección (recomendado)

Devolveremos la respuesta de autenticación a esta dirección URI después de autenticar correctamente al usuario. Este dato es opcional y se puede cambiar más tarde, pero se necesita un valor para la mayoría de los escenarios de autenticación.

Web

https://jwt.ms

Permisos

Azure AD B2C requiere que a esta aplicación se le hayan concedido los permisos `openid` y `offline_access`. Debe ser administrador de aplicaciones para conceder consentimiento del administrador (puede hacerlo más tarde desde el menú Permisos).

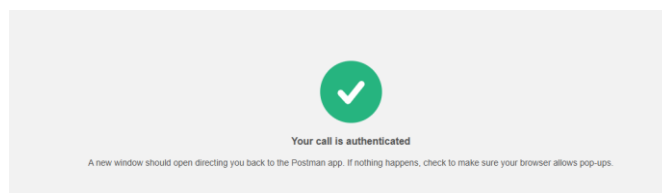
Conceda consentimiento del administrador a los permisos `openid` y `offline_access`.

Registre una aplicación en la que esté trabajando aquí. Integre aplicaciones de la galería y otras aplicaciones de fuera de la organización agregándolas desde [Aplicaciones empresariales](#).

Registro de la aplicación plataforma-cursos-api y configuración de la URI de redirección.

6.3 Flujo de usuario y emisión de JWT

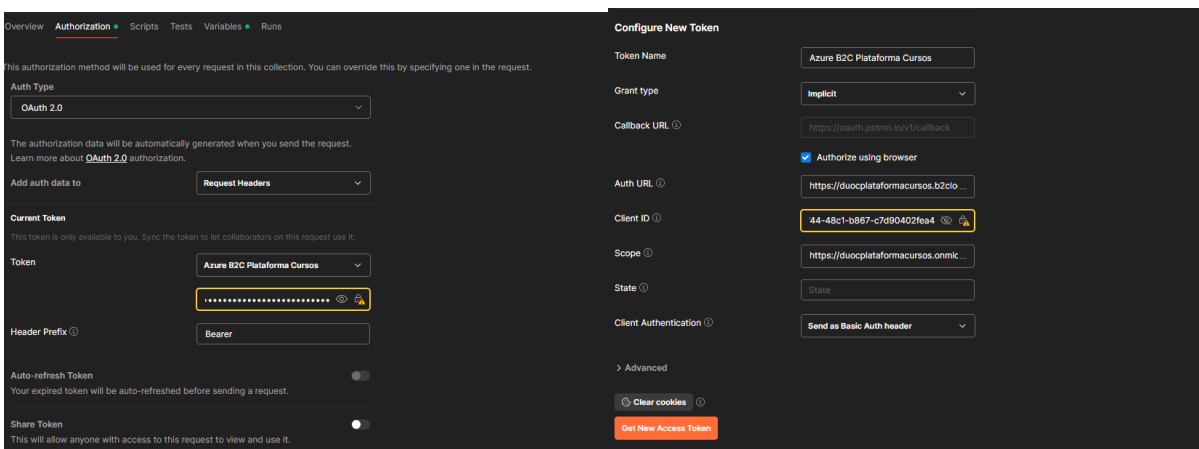
8. Crear el flujo B2C_1_susi_plataforma_cursos para registro e inicio de sesión.
9. Agregar los atributos requeridos para identificar al usuario y su rol.
10. Configurar el atributo personalizado extension_Rol con los valores ESTUDIANTE e INSTRUCTOR.
11. Ejecutar el flujo de usuario, autenticar una cuenta y obtener el token.
12. Validar el contenido del JWT, especialmente issuer, audience y extension_Rol.



Autenticación completada y retorno hacia Postman.



JWT decodificado con el rol ESTUDIANTE.



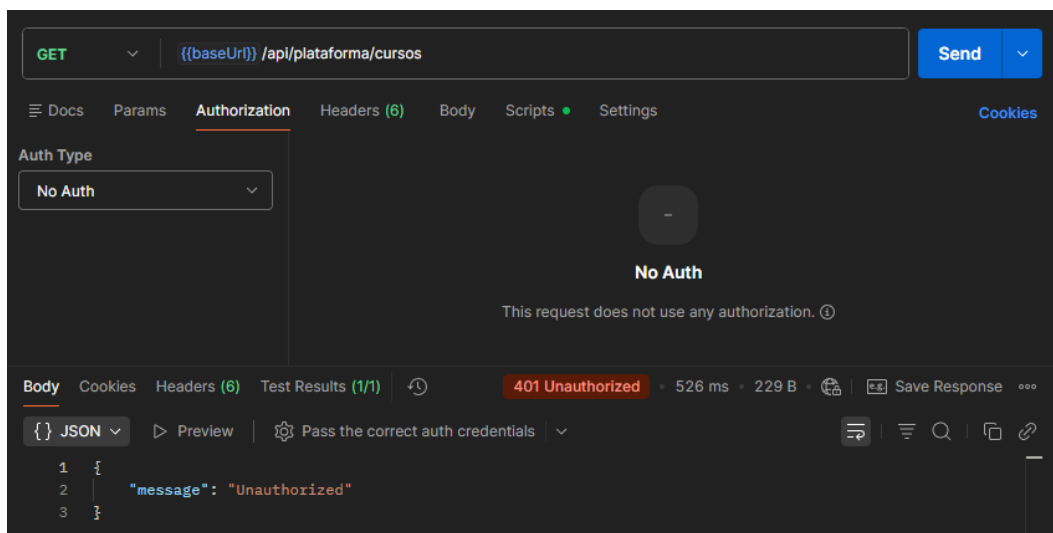
Configuración OAuth 2.0 de la colección Postman para obtener tokens de Azure AD B2C.

7. Seguridad del backend con Spring Security

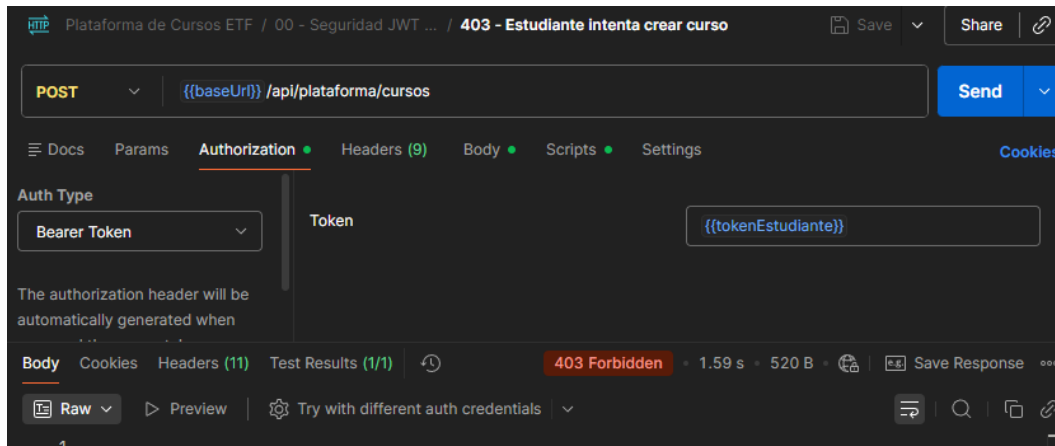
El BFF funciona como OAuth 2.0 Resource Server. SecurityConfig deshabilita sesiones, utiliza JWT y aplica autorización por método HTTP y ruta. El JwtAuthenticationConverter delega la extracción de roles en JwtRoleConverter, que interpreta extension_Rol.

Funcionalidad	ESTUDIANTE	INSTRUCTOR
Consultar cursos y exámenes	Permitido	Permitido
Crear, actualizar o desactivar cursos	Denegado	Permitido
Listar y descargar archivos	Permitido	Permitido
Subir, reemplazar o eliminar archivos	Denegado	Permitido
Publicar inscripción	Permitido	Denegado
Responder examen	Permitido	Denegado
Consultar notas del estudiante	Permitido	Denegado
Consultar o modificar notas del curso	Denegado	Permitido
Consumir DLQ	Denegado	Permitido

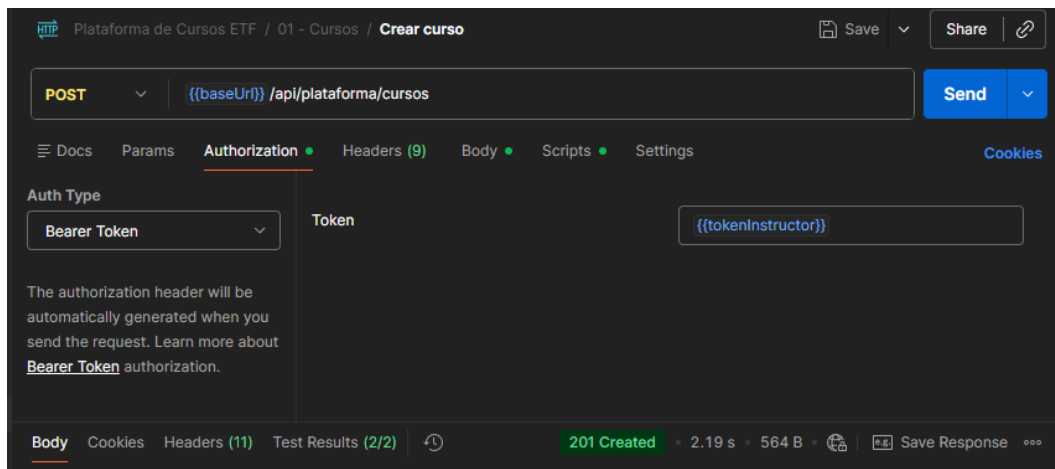
Las pruebas manuales verificaron 401 Unauthorized sin token, 403 Forbidden con un rol no autorizado y respuestas exitosas con el rol correcto. Esta doble protección se aplica en API Gateway y en Spring Security, evitando que una ruta expuesta públicamente pueda ejecutarse sin las autoridades requeridas.



Solicitud sin token rechazada con código 401 Unauthorized.



Intento de creación de curso con rol ESTUDIANTE rechazado con código 403 Forbidden.



Creación de curso autorizada para el rol INSTRUCTOR con código 201 Created.

8. Configuración de AWS API Gateway

Se utilizó una HTTP API denominada plataforma-cursos-api. API Gateway actúa como punto único de entrada y dirige todas las solicitudes al BFF. La URL pública utilizada para las pruebas fue <https://c93ehjbubk.execute-api.us-east-1.amazonaws.com>.

8.1 Creación del autorizador JWT

13. Ingresar a API Gateway > plataforma-cursos-api > Autorización.
14. Crear un autorizador de tipo JWT denominado azure-b2c-jwt.
15. Configurar el issuer del flujo B2C y la audiencia del registro de aplicación.
16. Guardar el autorizador y asociarlo a cada ruta publicada.

Tipo de autorizador
ID del autorizador
b5d86

Tipo de autorizador
JWT

Configuración del autorizador
Nombre
Escriba un nombre para el autorizador.

Origen de identidad
Escriba una expresión de selección que defina el origen del token.

URL del emisor
Escriba la URL del emisor de su proveedor de identidad, que suele encontrarse en el campo emisor del punto de enlace de metadatos conocido del servidor de autorización.

Audiencia
Escriba los ID de clientes registrados con proveedores de identidad o cualquier cadena arbitraria en la reclamación de público de JWT que el autorizador debe verificar.

Eliminar

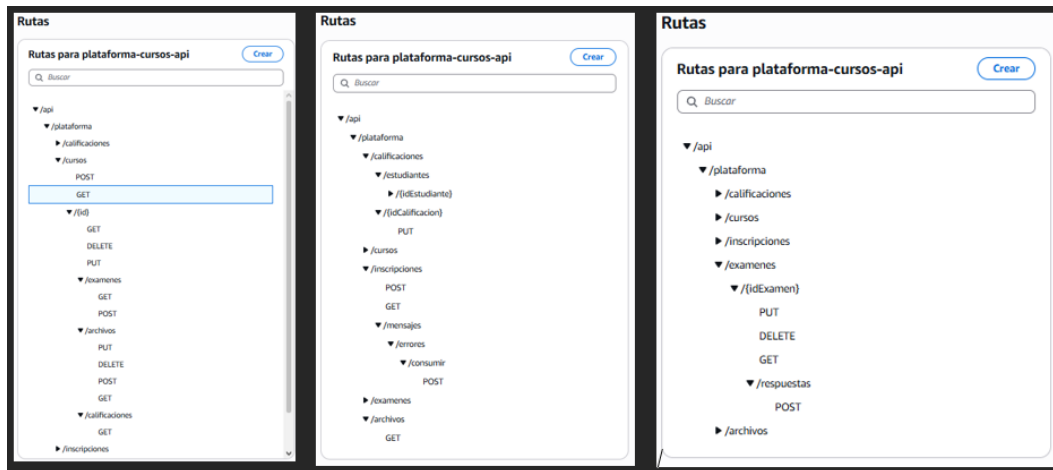
Agregar público

Cancelar Guardar

Configuración del autorizador JWT azure-b2c-jwt con issuer y audience de Azure AD B2C.

8.2 Registro de rutas e integración

17. Crear las rutas HTTP requeridas para cursos, inscripciones, archivos, exámenes y calificaciones.
18. Asignar a cada ruta la su respectiva integración HTTP orientada al BFF desplegado en EC2.
19. Asociar azure-b2c-jwt a todas las rutas funcionales.
20. Probar la API desde Postman utilizando la URL pública y Bearer Token.



Rutas configuradas en AWS API Gateway.

Rutas para plataforma-cursos-api

- ▼ /api
 - ▼ /plataforma
 - ▼ /calificaciones
 - ▼ /estudiantes
 - ▼ /idEstudiante

GET JWT Auth

Autorizador para la ruta GET /api/plataforma/calificaciones/estudiantes/{idEstudiante}

Desconectar autorizador

Editar autorizador

Nombre de autorizador azure-b2c-jwt	Tipo de autorizador JWT	ID del autorizador i5id6
---	-----------------------------------	------------------------------------

Origen de identidad
Cuando se invoca a este autorizador, API Gateway utilizará esta expresión de selección para determinar el origen del token

\$request.header.Authorization

Emisor
El URI del emisor del proveedor de identidad
https://duocplataformacursos.b2clogin.com/tpf46585c26-bd19-4e8d-9811-4811861aa451/b2c_1_susi_plataforma_cursos/v2.0/

Audencia
El público asociado con este autorizador

- ff52fc0e-51de-4bbc-a952-992918b6e6bb

Ámbitos de autorización

Especifique uno o varios ámbitos necesarios para obtener acceso a esta ruta de API.
No hay ámbitos configurados para esta ruta

Agregar ámbito

Guardar

Autorizador JWT asociado a las rutas de API Gateway.

Rutas para plataforma-cursos-api

- ▼ /api
 - ▼ /plataforma
 - ▼ /calificaciones
 - ▼ /estudiantes
 - ▼ /idEstudiante

GET HTTP GET

Detalles de integración para la ruta

Desconectar integración

Gestionar integración

URI del HTTP GET http://98.86.187.63:8080/api/plataforma/calificaciones/estudiantes/{idEstudiante}	ID de integración 4gb2r2l
---	-------------------------------------

Descripción
-

Tiempo de espera
El número de milisegundos que API Gateway debe esperar por una respuesta de la integración antes de que se agote el tiempo de espera.
30000

Solicitar asignación de parámetros No configurado	Asignaciones de parámetros de respuesta No configurado
---	--

Integración HTTP entre API Gateway y el BFF.

9. Configuración de RabbitMQ y Docker

RabbitMQ se despliega mediante Docker Compose utilizando la imagen rabbitmq:3-management. La consola se publica en el puerto 15672, mientras el puerto AMQP 5672 se utiliza dentro de la red Docker. Las credenciales se inyectan mediante variables de entorno y el volumen rabbitmq-data conserva la información del broker.

```

rabbitmq:
  image: rabbitmq:3-management
  container_name: plataforma-rabbitmq
  hostname: rabbitmq
  restart: unless-stopped

  environment:
    RABBITMQ_DEFAULT_USER: ${RABBITMQ_USERNAME}
    RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASSWORD}

  volumes:
    - rabbitmq_data:/var/lib/rabbitmq

  ports:
    - "15672:15672"

  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 20s

  networks:
    - plataforma-network

```

Configuración del contenedor RabbitMQ en Docker Compose.

Componente	Nombre	Propósito
Exchange principal	inscripciones.exchange	Recibe las solicitudes de inscripción.
Cola principal	inscripciones.queue	Almacena solicitudes pendientes de consumo.
Routing key principal	inscripciones.nueva	Enruta mensajes desde el exchange hacia la cola principal.
Dead Letter Exchange	inscripciones.dlx	Recibe los mensajes rechazados o fallidos.
Cola de errores	inscripciones.error.queue	Conserva mensajes que no pudieron procesarse.
Routing key de error	inscripciones.error	Enruta mensajes fallidos hacia la DLQ.

Queues

▼ All queues (2)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	inscripciones.error.queue	classic	D	running	0	0	0				
/	inscripciones.queue	classic	D DLX DLK	running	0	0	0				

► Add a new queue

Colas configuradas en RabbitMQ Management.

Exchanges

▼ All exchanges (9)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Virtual host	Name	Type	Features	Message rate in	Message rate out	+/-
/	(AMQP default)	direct	D			
/	amq.direct	direct	D			
/	amq.fanout	fanout	D			
/	amq.headers	headers	D			
/	amq.match	headers	D			
/	amq.rabbitmq.trace	topic	D I			
/	amq.topic	topic	D			
/	inscripciones.dlx	direct	D			
/	inscripciones.exchange	direct	D			

► Add a new exchange

Exchanges configurados en RabbitMQ Management.

10. Productor, consumidor y Dead Letter Queue

10.1 Endpoint productor

El endpoint POST /api/plataforma/inscripciones recibe un IncripcionMessage y el BFF lo envía a inscripciones-service. IncripcionProducer utiliza RabbitTemplate.convertAndSend para publicar el mensaje en inscripciones.exchange usando la routing key inscripciones.nueva. La respuesta inmediata informa EN_PROCESO, desacoplando la recepción del procesamiento.

```
public void publicar(IncripcionMessage mensaje) {  
    rabbitTemplate.convertAndSend(  
        RabbitMqConfig.INScripciones_EXCHANGE,  
        RabbitMqConfig.INScripciones_ROUTING_KEY,  
        mensaje  
    );  
}
```

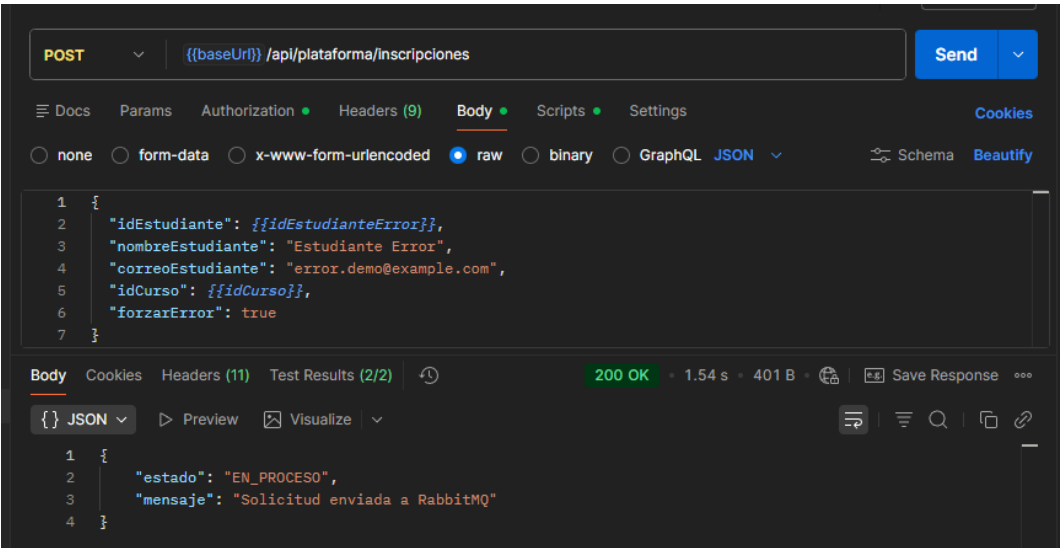
Publicación de mensajes mediante RabbitTemplate.

10.2 Consumidor automático

IncripcionConsumer escucha inscripciones.queue mediante @RabbitListener. El consumidor transforma el mensaje en IncripcionRequest, registra la inscripción en Oracle y confirma el flujo. Para la demostración se mantuvo una espera controlada de diez segundos, lo que permite observar el mensaje antes y después del consumo.

10.3 Flujo de error y DLQ

Cuando forzarError es true, el consumidor lanza una excepción. La cola principal está configurada con x-dead-letter-exchange y x-dead-letter-routing-key, por lo que RabbitMQ deriva el mensaje a inscripciones.error.queue. El endpoint POST /api/plataforma/inscripciones/mensajes/errores/consumir utiliza RabbitTemplate.receiveAndConvert para retirar y mostrar un mensaje desde la DLQ.



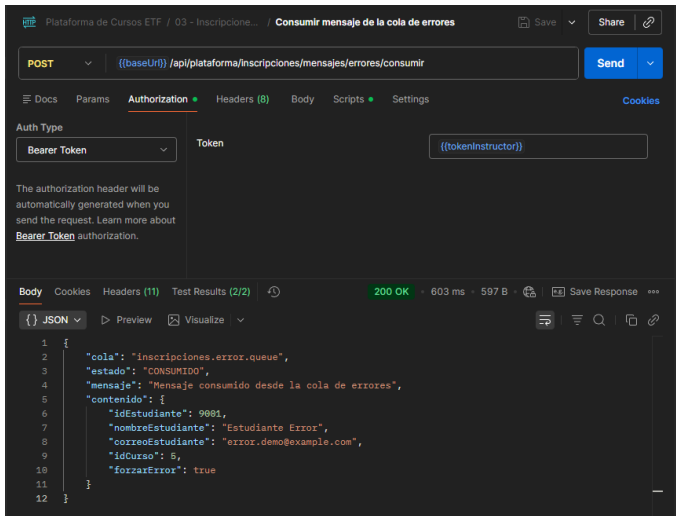
Publicación de una solicitud de inscripción con error forzado.

Overview					Messages			Message rates			
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	inscripciones.error.queue	classic	D	running	0	0	0				
/	inscripciones.queue	classic	D DLX DLK	running	0	1	1	0.00/s	0.00/s	0.00/s	

Mensaje en procesamiento dentro de la cola principal.

Overview					Messages			Message rates			
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	inscripciones.error.queue	classic	D	running	1	0	1				
/	inscripciones.queue	classic	D DLX DLK	running	0	0	0	0.00/s	0.00/s	0.00/s	

Mensaje redirigido a la cola de errores mediante DLQ.



Consumo manual de un mensaje desde la cola de errores.

Overview					Messages			Message rates			
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	inscripciones.error.queue	classic	D	running	0	0	0			0.00/s	0.00/s
/	inscripciones.queue	classic	D DLX DLK	running	0	0	0	0.00/s	0.00/s	0.00/s	

Estado final de las colas después del consumo del mensaje de error.

11. Almacenamiento Cloud con Amazon S3

Amazon S3 se utiliza para almacenar los materiales asociados a cada curso. El bucket configurado es duoc-plataforma-cursos en us-east-1. cursos-service usa AWS SDK para subir, listar, reemplazar, descargar y eliminar objetos. Las claves se organizan por curso, lo que facilita la recuperación mediante el prefijo cursos/{idCurso}/.

11.1 Creación y configuración del bucket

21. Crear un bucket de uso general en us-east-1 con nombre global único.
22. Deshabilitar ACL para mantener propiedad de objetos en la cuenta.
23. Configurar el acceso requerido por el backend mediante credenciales y políticas IAM.
24. Mantener cifrado del lado del servidor con claves administradas por Amazon S3.
25. Inyectar AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY y AWS_SESSION_TOKEN en cursos-service durante el despliegue.

Crear bucket [Información](#)
Los buckets son contenedores de datos almacenados en S3.

Configuración general

Región de AWS
EE.UU. Este (Norte de Virginia) us-east-1

Tipo de bucket | [Información](#)
☒ **Uso general**
Recomendado para la mayoría de los casos de uso y patrones de acceso. Los buckets de uso general son del tipo de bucket de S3 original. Permiten una combinación de clases de almacenamiento que almacenan objetos de forma redundante en múltiples zonas de disponibilidad.
☐ **Directorio**
Recomendado para casos de uso de baja latencia. Estos buckets utilizan únicamente la clase de almacenamiento S3 Express One Zone, que proporciona un procesamiento más rápido de los datos dentro de una única zona de disponibilidad.

Espacio de nombres del bucket
Elija el espacio de nombres en el que quiere crear su bucket. [Más información](#)
☒ **Espacio de nombres global**
De forma predeterminada, S3 crea buckets de uso general en el espacio de nombres global.
☐ **Espacio de nombres regional de la cuenta (recomendado)**
Los buckets de uso general creados en el espacio de nombres regional de su cuenta son exclusivos de su cuenta. Estos buckets nunca se pueden crear con otra cuenta de AWS.

Nombre del bucket | [Información](#)
duoc-plataforma-cursos
Los nombres de los buckets deben tener entre 3 y 63 caracteres y ser únicos dentro del espacio de nombres global. Los nombres de los buckets también deben empezar y terminar con una letra o un número. Los caracteres válidos son a-z, 0-9, puntos (.) y guiones (-). [Más información](#)

Copiar la configuración del bucket existente: opcional
Solo se copia la configuración del bucket en los siguientes ajustes.
[Elegir el bucket](#)
Formato: s3://bucket/prefijo

Creación del bucket duoc-plataforma-cursos en la región us-east-1.

Propiedad de objetos

Información

Controle la propiedad de los objetos escritos en este bucket desde otras cuentas de AWS y el uso de listas de control de acceso (ACL). La propiedad de los objetos determina quién puede especificar el acceso a los objetos.

Propiedad del objeto

☒ ACL deshabilitadas (recomendado)

Todos los objetos de este bucket son propiedad de esta cuenta. El acceso a este bucket y sus objetos se especifica solo mediante políticas.

☐ ACL habilitadas

Los objetos de este bucket pueden ser propiedad de otras cuentas de AWS. El acceso a este bucket y sus objetos se puede especificar mediante ACL.

Propiedad del objeto

Aplicada al propietario del bucket

Configuración de bloqueo de acceso público para este bucket

Se concede acceso público a los buckets y objetos a través de listas de control de acceso (ACL), políticas de bucket, políticas de puntos de acceso o todas las anteriores. A fin de garantizar que se bloquee el acceso público a todos sus buckets y objetos, active Bloquear todo el acceso público. Esta configuración se aplica exclusivamente a este bucket y a sus puntos de acceso. AWS recomienda activar Bloquear todo el acceso público, pero, antes de aplicar cualquiera de estos ajustes, asegúrese de que las aplicaciones funcionarán correctamente sin acceso público. Si necesita cierto nivel de acceso público a los buckets u objetos, puede personalizar la configuración individual a continuación para adaptarla a sus casos de uso de almacenamiento específicos. [Más información](#)

☐ Bloquear todo el acceso público

Activar esta configuración equivale a activar las cuatro opciones que aparecen a continuación. Cada uno de los siguientes ajustes son independientes entre sí.

☐ Bloquear el acceso público a buckets y objetos concedido a través de nuevas listas de control de acceso (ACL)

S3 bloqueará los permisos de acceso público aplicados a objetos o buckets agregados recientemente, y evitará la creación de nuevas ACL de acceso público para buckets y objetos existentes. Esta configuración no cambia los permisos existentes que permiten acceso público a los recursos de S3 mediante ACL.

☐ Bloquear el acceso público a buckets y objetos concedido a través de cualquier lista de control de acceso (ACL)

S3 ignorará todas las ACL que concedan acceso público a buckets y objetos.

☐ Bloquear el acceso público a buckets y objetos concedido a través de políticas de bucket y puntos de acceso públicos nuevos

S3 bloqueará las nuevas políticas de buckets y puntos de acceso que concedan acceso público a buckets y objetos. Esta configuración no afecta a las políticas ya existentes que permiten acceso público a los recursos de S3.

☐ Bloquear el acceso público y entre cuentas a buckets y objetos concedido a través de cualquier política de bucket y puntos de acceso pública

S3 ignorará el acceso público y entre cuentas en el caso de buckets o puntos de acceso que tengan políticas que concedan acceso público a buckets y objetos.

⚠ Desactivar el bloqueo de todo acceso público puede provocar que este bucket y los objetos que contiene se vuelvan públicos

AWS recomienda que active la opción para bloquear todo el acceso público, a menos que se requiera acceso público para casos de uso específicos y verificados, como el alojamiento de sitios web estáticos.

☐ Reconozco que la configuración actual puede provocar que este bucket y los objetos que contiene se vuelvan públicos.

Control de versiones de buckets

El control de versiones es una forma de mantener múltiples variantes de un objeto dentro del mismo bucket. Puede utilizar el control de versiones para conservar, recuperar y restaurar todas las versiones de los objetos almacenados en su bucket de Amazon S3. Con el control de versiones, puede recuperarse con facilidad de las acciones involuntarias de los usuarios y de los errores en las aplicaciones. [Más información](#)

Control de versiones de buckets

☒ Desactivar

☐ Habilitar

Propiedad de objetos y configuración de acceso.

Etiquetas - opcional

Puede usar etiquetas de bucket para analizar los permisos de un bucket, además de administrarlos y especificarlos. [Más información](#)

🔗 Puede usar las APIs s3:ListTagForResource, s3:TagResource y s3:UntagResource para administrar las etiquetas en los buckets de uso general de S3 para el control de acceso, además de la asignación de costos y la organización de los recursos. Para garantizar una transición fluida, proporcione permisos para realizar las acciones s3:ListTagForResource, s3:TagResource y s3:UntagResource. [Más información](#)

No hay etiquetas asociadas a este bucket.

Agregar nueva etiqueta

Puede agregar hasta 50 etiquetas.

Cifrado predeterminado

Información

El cifrado del lado del servidor se aplica automáticamente a los nuevos objetos almacenados en este bucket.

Tipo de cifrado

Información

Pregunte sus objetos con dos casos de cifrado independientes. Para obtener más información sobre los precios, consulte DISE-KMS pricing (Precios de DISE-KMS) en la pestaña Storage (Almacenamiento) de la [página de precios de Amazon S3](#).

☒ Cifrado del servidor con claves administradas de Amazon S3 (SSE-S3)

☐ Cifrado del servidor con claves de AWS Key Management Service (SSE-KMS)

☐ Cifrado de doble capa del servidor con claves de AWS Key Management Service (DISE-KMS)

Clave de bucket

El uso de una clave de bucket de S3 para SSE-KMS reduce los costos de cifrado al reducir las llamadas a AWS KMS. Las claves de bucket de S3 no son compatibles con DISE-KMS. [Más información](#)

☐ Desactivar

☒ Habilitar

▼ Configuración avanzada

Bloqueo de objetos

Almacene objetos mediante un modelo de escritura única, lectura múltiple (WORM, write-once-read-many) para evitar que se eliminen o sobrescriban objetos durante un período de tiempo fijo o de manera indefinida. El bloqueo de objetos solo funciona en buckets con control de versiones. [Más información](#)

☒ Desactivar

☐ Habilitar

Permitir permanentemente bloquear los objetos de este bucket. Después de la creación del bucket, se requiere una configuración adicional de bloqueo de objetos en los detalles del bucket para proteger sus objetos y que no se eliminen o sobrescriban.

🔗 El bloqueo de objetos solo funciona en buckets con control de versiones. Al habilitar el bloqueo de objetos, se habilita automáticamente el control de versiones.

🔗 Después de crear el bucket, puede cargar archivos y carpetas, y configurar ajustes adicionales en él.

Cancelar

Crear bucket

Cifrado SSE-S3 y configuración avanzada.

Método	Endpoint	Acción
POST	/api/plataforma/cursos/{idCurso}/archivos	Subir material.
GET	/api/plataforma/cursos/{idCurso}/archivos	Listar materiales del curso.
PUT	/api/plataforma/cursos/{idCurso}/archivos?key=...	Reemplazar un objeto existente.
GET	/api/plataforma/archivos?key=...	Descargar un objeto.
DELETE	/api/plataforma/cursos/{idCurso}/archivos?key=...	Eliminar un objeto.

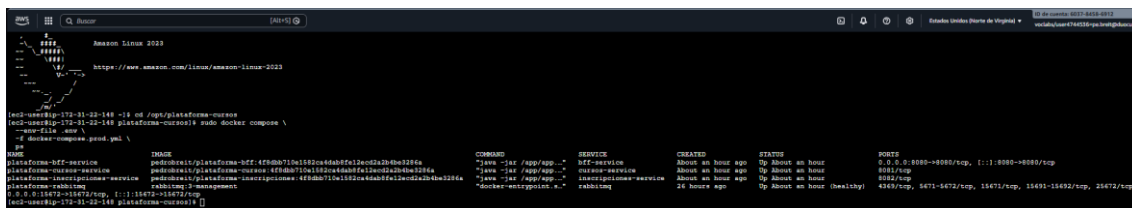
12. Pipeline CI/CD y despliegue en Amazon EC2

El workflow `.github/workflows/main.yml` se ejecuta cuando se realiza push a main. La primera etapa prueba los tres microservicios con Java 21. Después construye imágenes Docker para `bff-service`, `cursos-service` e `inscripciones-service`, las publica en Docker Hub con la etiqueta del commit y latest, y finalmente despliega en Amazon EC2.



Ejecución exitosa de pruebas, construcción de imágenes y despliegue en EC2.

26. Ejecutar pruebas Maven para cada microservicio en una matriz de GitHub Actions.
27. Construir imágenes linux/amd64 mediante Docker Buildx.
28. Publicar pedrobreit/plataforma-bff, pedrobreit/plataforma-cursos y pedrobreit/plataforma-inscripciones.
29. Obtener la IP temporal del runner y autorizar SSH en el Security Group.
30. Reconstruir y validar el wallet Oracle desde ORACLE_WALLET_B64.
31. Copiar docker-compose.prod.yml, variables privadas y wallet a EC2.
32. Instalar o validar Docker Compose, descargar las imágenes y ejecutar docker compose up -d.
33. Validar el BFF solicitando `/api/plataforma/cursos` sin token. El HTTP 401 confirma que la aplicación inició y la seguridad está activa.
34. Revocar la regla SSH temporal y eliminar archivos sensibles del runner.

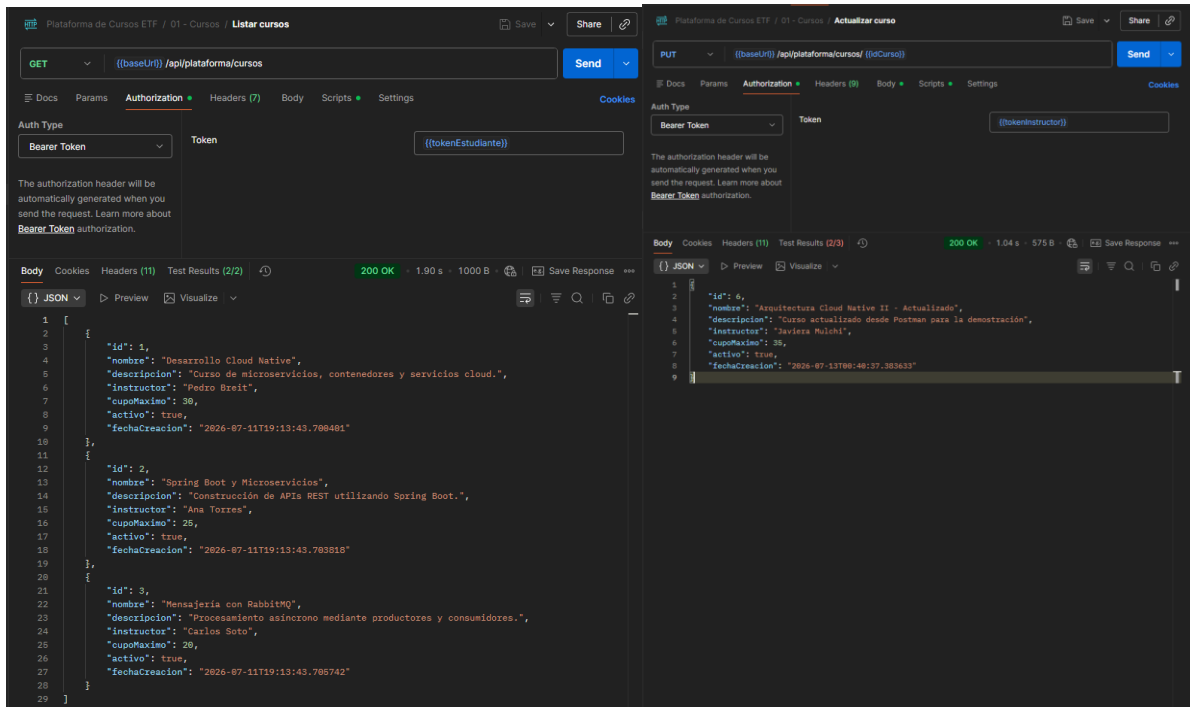


Contenedores desplegados en Amazon EC2 mediante Docker Compose.

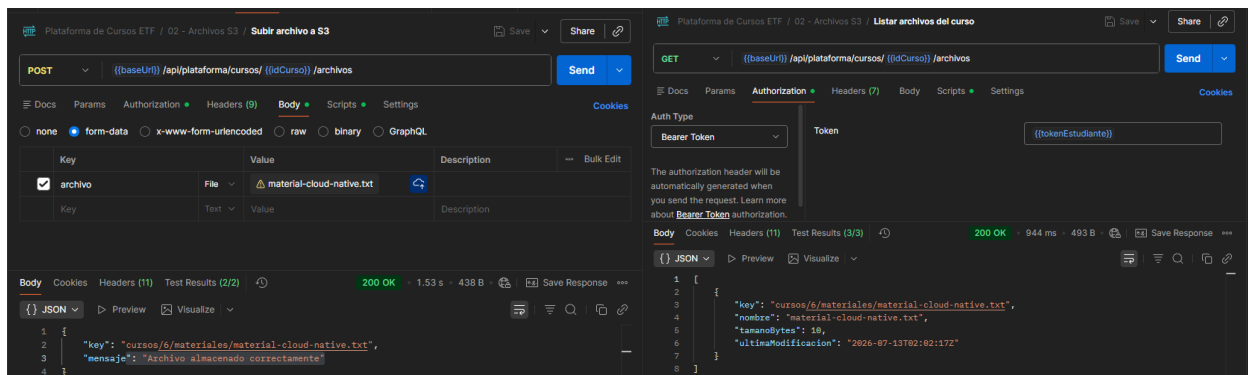
13. Validación funcional con Postman

Se creó una colección organizada por seguridad, cursos, archivos S3, inscripciones y RabbitMQ, exámenes, calificaciones y limpieza. Las variables del ambiente almacenan baseUrl, tokens, idCurso, idExamen, idCalificacion, idEstudiante y archivoKey. Las pruebas manuales confirmaron el comportamiento esperado.

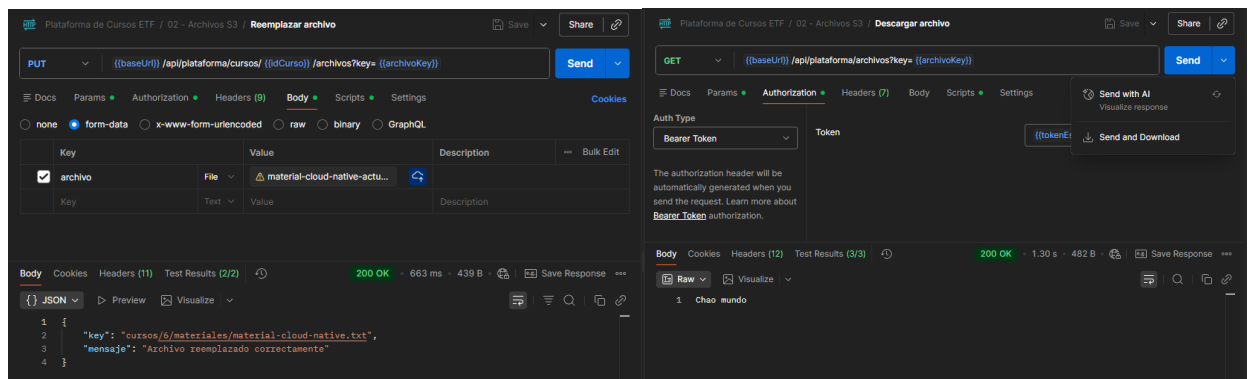
Prueba	Resultado observado	Validación
Solicitud sin token	401 Unauthorized	API protegida.
ESTUDIANTE intenta crear curso	403 Forbidden	Regla de rol correcta.
Crear curso	201 Created	Persistencia y variable idCurso.
Listar / actualizar curso	200 OK	Consulta y edición.
Subir / listar / reemplazar / descargar archivo	200 OK	Integración S3 funcional.
Eliminar archivo	204 No Content	Eliminación en S3.
Publicar inscripción	200 OK + EN_PROCESO	Productor RabbitMQ.
Consultar inscripción tras espera	200 OK + CONFIRMADA	Consumidor automático + Oracle.
Publicar error y consumir DLQ	200 OK + CONSUMIDO	Dead Letter Queue funcional.
Crear examen / responder	201 Created	Examen y calificación automática.
Consultar y actualizar nota	200 OK	Gestión de calificaciones.
Eliminar examen / curso	204 No Content	Desactivación lógica.



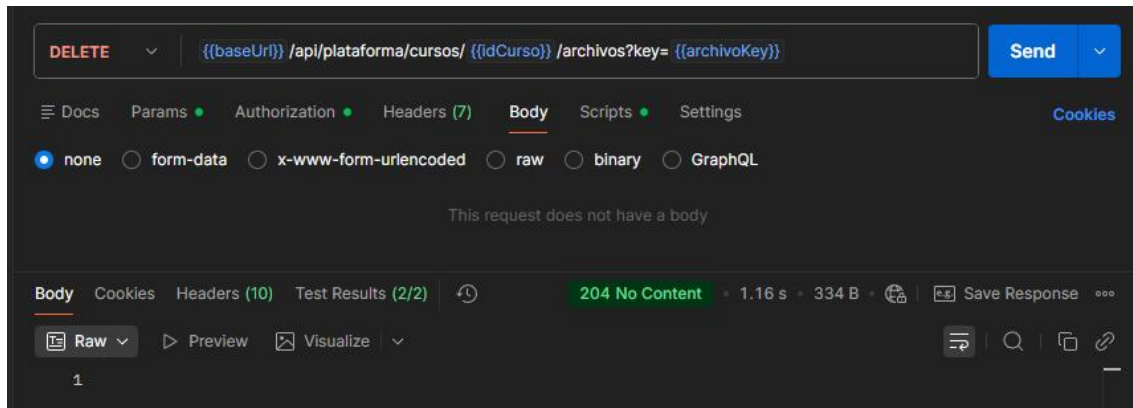
Consulta y actualización de cursos mediante la API.



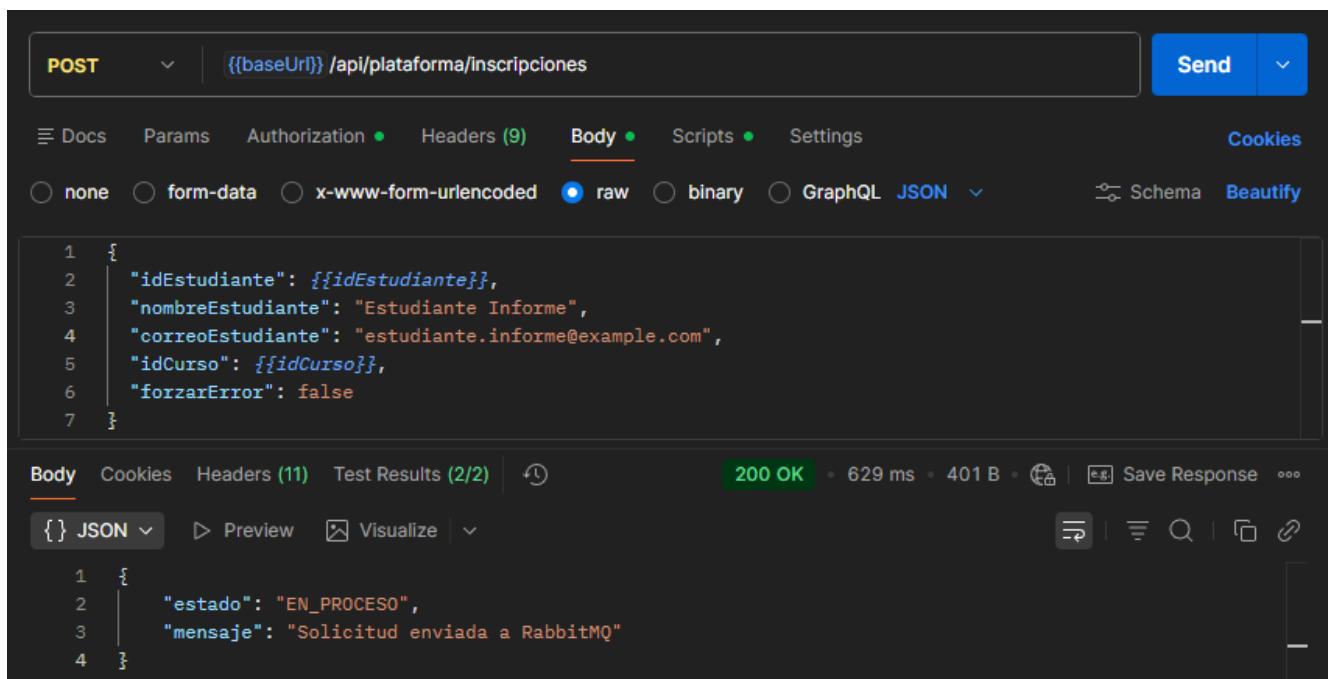
Carga y listado de archivos almacenados en Amazon S3.



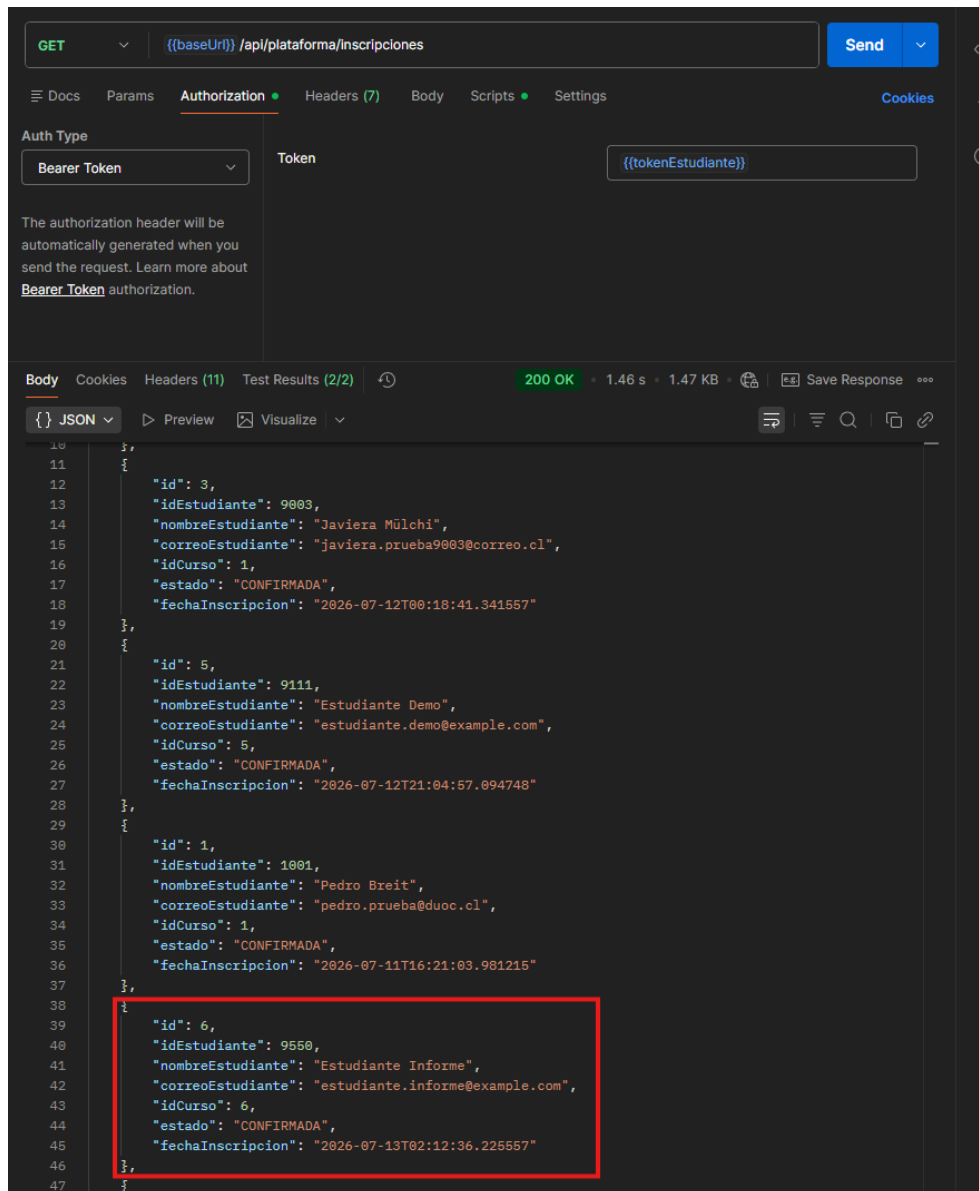
Actualización y descarga de archivos almacenados en Amazon S3.



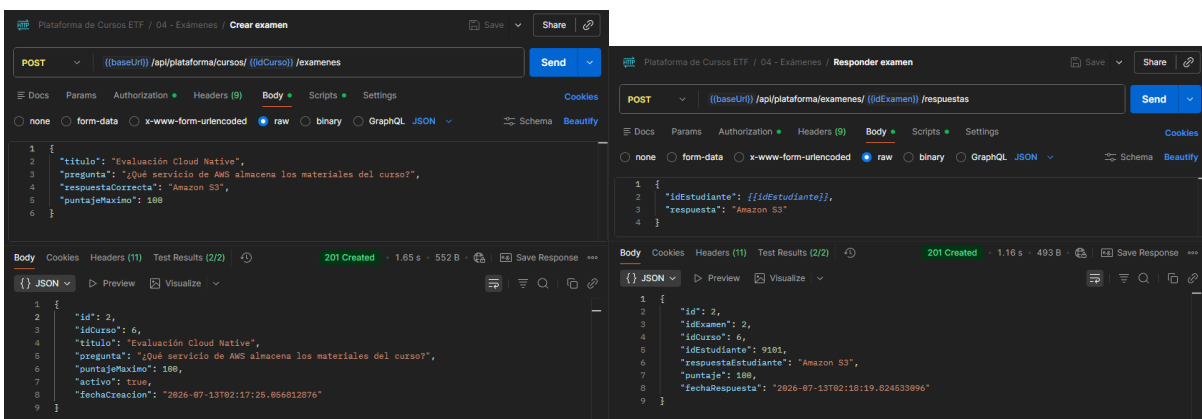
Eliminación de archivos almacenados en Amazon S3.



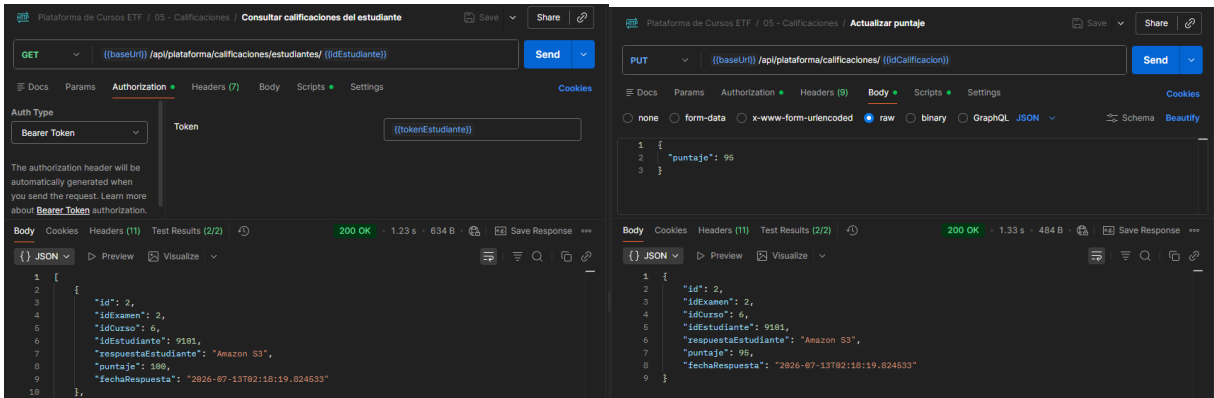
Publicación de una solicitud de inscripción mediante RabbitMQ.



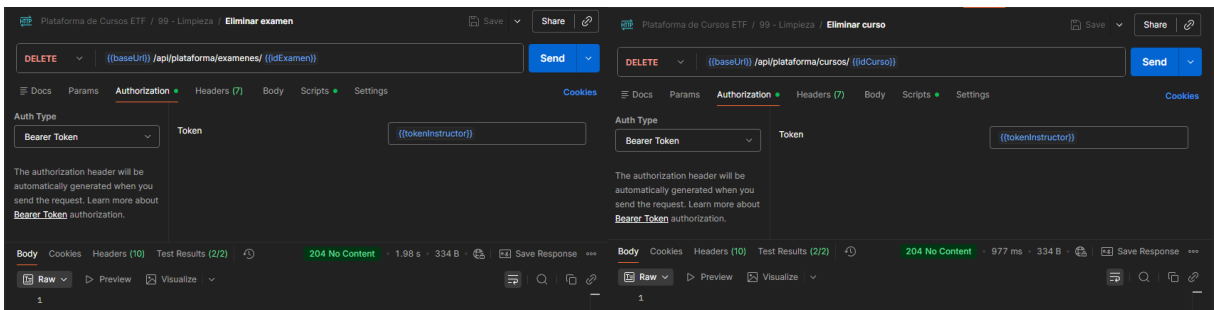
Consulta de inscripción procesada de forma asíncrona.



Creación de un examen y registro de la respuesta de un estudiante.



Consulta y actualización de la calificación de un estudiante.



Desactivación lógica de un examen y un curso de la plataforma.

Plataforma de Cursos ETF			Run	Fork	0	Watch	0	Share	
Overview Authorization Scripts Variables Runs									
Variable	Value								
baseUrl	https://c93ehjbubk.execute-api.us-east-1.amazonaws.com								
tokenEstudiante	eyJhbGciOiJSUzI1NiIsImtpZCI6Ilg1ZVhrNHh5b2pORnVtMWtsMlI0djhkbE5...								
tokenInstructor	eyJhbGciOiJSUzI1NiIsImtpZCI6Ilg1ZVhrNHh5b2pORnVtMWtsMlI0djhkbE5...								
idCurso	6								
idExamen	2								
idCalificacion	2								
☒ IdEstudianteError	9001								Share
archivoKey	cursos/6/materiales/material-cloud-native.txt								

Variables de entorno utilizadas en Postman para las pruebas de la API.

14. Catálogo de endpoints

Método	Ruta	Rol	Descripción
GET	/api/plataforma/cursos	Ambos	Listar cursos activos.
POST	/api/plataforma/cursos	INSTRUCTOR	Crear curso.
GET	/api/plataforma/cursos/{id}	Ambos	Consultar curso.
PUT	/api/plataforma/cursos/{id}	INSTRUCTOR	Actualizar curso.
DELETE	/api/plataforma/cursos/{id}	INSTRUCTOR	Desactivar curso.
POST	/api/plataforma/cursos/{id}/archivos	INSTRUCTOR	Subir archivo.
GET	/api/plataforma/cursos/{id}/archivos	Ambos	Listar archivos.
PUT	/api/plataforma/cursos/{id}/archivos	INSTRUCTOR	Reemplazar archivo.
DELETE	/api/plataforma/cursos/{id}/archivos	INSTRUCTOR	Eliminar archivo.
GET	/api/plataforma/archivos	Ambos	Descargar archivo.
GET	/api/plataforma/inscripciones	ESTUDIANTE	Listar inscripciones.
POST	/api/plataforma/inscripciones	ESTUDIANTE	Publicar inscripción en RabbitMQ.
POST	/api/plataforma/inscripciones/mensajes/errores/consumir	INSTRUCTOR	Consumir DLQ.
GET	/api/plataforma/cursos/{id}/exámenes	Ambos	Listar exámenes.

POST	/api/plataforma/cursos/{id}/examenes	INSTRUCTOR	Crear examen.
GET	/api/plataforma/examenes/{idExamen}	Ambos	Consultar examen.
PUT	/api/plataforma/examenes/{idExamen}	INSTRUCTOR	Actualizar examen.
DELETE	/api/plataforma/examenes/{idExamen}	INSTRUCTOR	Desactivar examen.
POST	/api/plataforma/examenes/{idExamen}/respuestas	ESTUDIANTE	Responder examen.
GET	/api/plataforma/calificaciones/estudiantes/{idEstudiante}	ESTUDIANTE	Consultar calificaciones.
GET	/api/plataforma/cursos/{id}/calificaciones	INSTRUCTOR	Consultar notas del curso.
PUT	/api/plataforma/calificaciones/{idCalificacion}	INSTRUCTOR	Modificar puntaje.

15. Conclusiones y repositorio

La plataforma implementada cumple con el caso solicitado al integrar microservicios Spring Boot, seguridad basada en Azure AD B2C, administración de endpoints mediante AWS API Gateway, mensajería asíncrona con RabbitMQ, almacenamiento de archivos en Amazon S3, persistencia en Oracle Autonomous Database y despliegue automatizado en Amazon EC2. Las pruebas realizadas validaron tanto los flujos exitosos como los controles de seguridad y el manejo de errores. La solución demostró autenticación, autorización por roles, comunicación entre microservicios, persistencia, almacenamiento Cloud, publicación y consumo de mensajes, DLQ, exámenes y calificaciones.

Repositorio GitHub: <https://github.com/PedroBreit/DUOC-DCN-EFT>



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.